AI4DueDil — Metrics Reference

Domain: Compliance due diligence in insurance (AXA France) **System type:** RAG-based ordinal classification with human expert ground truth **Verdict space:** `compliant` / `partially_compliant` / `non_compliant`

Domain context

This is not a standard QA or information-retrieval task. Several properties make it distinct from general RAG evaluation:

- **Ordinal 3-class verdict** — the three labels have a natural order and non-symmetric error costs. Predicting `compliant` for a true `non_compliant` is categorically worse than predicting `partially_compliant`.
- **Asymmetric error costs** — a missed non-compliant (false positive on compliant) allows a risky subcontractor through the due diligence gate. A false alarm (non-compliant when actually compliant) causes rework but no regulatory exposure. All thresholds and metric weights should reflect this asymmetry.
- **Two evidence paths** — some exigences require hard documentary proof (ISO certificates, audit reports, signed policies). Others are inherently declarative (vendor commits to notification procedures, employees attended training). Retrieval metrics only make sense on the document path.
- **Regulatory explainability mandate** — the EU AI Act (Article 13) requires high-risk AI decisions in financial services to be traceable to a specific source. Every verdict must cite a file, a page, or a direct quote from the representative's declaration.
- **Messy proof corpus** — proof folders contain merged 80-page PDFs, irrelevant documents, OCR noise, and incomplete mappings between what the representative claimed to provide and what was actually uploaded. Retrieval must be robust to noise, not just accurate on clean inputs.

Error asymmetry rule

> A missed non-compliant (the AI labels a true `non_compliant` as `compliant` or `partially_compliant`) is categorically more costly than a false alarm.

This principle governs:

- How the weighted Cohen's κ penalty matrix is defined (distance 2 for `compliant` ↔ `non_compliant`, distance 1 for adjacent pairs)

- Which confidence threshold triggers mandatory human review (lower threshold on predicted `compliant` than on predicted `non_compliant`)
- Which metric is used as the primary production acceptance criterion (false negative rate on `non_compliant`, not overall accuracy)

Implementation status

Status	Count
Implemented in `utils/evaluation.py`	9
Not yet implemented	9
Total metrics in this reference	18

Metric tiers

Priority	Meaning
Critical	Production blocker — system cannot go live without this
Important	Next sprint — significant impact on trust or safety
Nice to have	Useful but not blocking

Layer 1 — Retrieval quality

Applied only to `evidence_type == document_required` rows. Skip for `oral_implicit`.

Context Recall

Priority: Critical | **Status:** Implemented | **Path:** Document path only

What it measures: The fraction of expected evidence tokens that appear in the retrieved chunks. Answers the question: did we retrieve the page that actually contains the proof?

Why it matters in this domain: A missed ISO certificate page or a missed PCA document means the LLM verdict is formed without the key evidence. In compliance, recall failures are catastrophic — every relevant page must surface. A system with 0.95 context precision but 0.60

context recall will systematically miss non-compliant subcontractors whose proof is buried in large merged PDFs.

Current implementation: Token-level overlap between the retrieved chunks and the content of `human_notes` (the expert's free-text annotation). The human_notes field acts as a proxy for "expected evidence text".

Upgrade path: Replace token overlap with an LLM-as-judge call: prompt a second LLM to assess whether the retrieved context semantically covers the evidence described in human_notes. This handles paraphrase and French-language variation better than token overlap.

Thresholds:

- ▌ 0.85: good, strategy is finding the right pages
- 0.70–0.85: acceptable, monitor for regression
- < 0.70: retrieval strategy likely has a structural gap (BM25 keyword mismatch, chunking too coarse, or the relevant content is not in the indexed files)

These metrics dont mean anything if the business teams find that the system doesnt work as expected

Context Precision

Priority: Critical | **Status:** Implemented | **Path:** Document path only

What it measures: The fraction of retrieved chunks that are actually relevant to the expected evidence. Answers: are the retrieved chunks useful or are they noise?

Why it matters in this domain: Proof folders contain documents like recipe sections inside merged admin PDFs, expired attestations from prior years, and unrelated contractual clauses. Low precision means the LLM verdict prompt is filled with irrelevant context, which increases hallucination risk and causes the LLM to either ignore the noise (wasted tokens) or synthesise a plausible-sounding verdict from unrelated material.

Current implementation: Token overlap between each retrieved chunk's text and the expected evidence (human_notes or ai_evidence). A chunk with overlap > 0.20 is counted as relevant.

Upgrade path: Use a cross-encoder relevance scorer (the same model as the `reranker` strategy) to score each chunk against the exigence, rather than against the human_notes. This evaluates retrieval quality independently of the human annotation.

Thresholds:

- ▌ 0.60: good — most retrieved chunks are on-topic
- 0.30–0.60: acceptable — some noise, LLM can usually filter it
- < 0.30: problematic — LLM is working with mostly irrelevant context; flag for manual review if confidence is also high (see Overreliance metric)

MRR — Mean Reciprocal Rank

Priority: Important | **Status:** Not implemented | **Path:** Document path only

What it measures: The reciprocal of the rank of the first relevant chunk in the retrieved list. MRR = 1.0 means the relevant chunk is ranked first. MRR = 0.25 means it appears in position 4.

Why it matters in this domain: The LLM verdict node receives retrieved pages in ranked order. In practice, LLMs tend to weight earlier context more heavily (position bias, confirmed in multiple studies including Liu et al. 2023). If the most relevant page — the ISO certificate — is ranked 4th, the LLM may not weight it appropriately even if it technically "sees" it.

This is especially important for the `naive_rag` and `hyde` strategies, which have no re-ranking step and rely entirely on cosine distance for ordering.

Formula:

```
MRR = (1 / N) * sum(1 / rank_i)
```

where `rank_i` is the position of the first relevant chunk for row `i`, and N is the total number of evaluated rows.

To implement: Requires per-row ground-truth annotation of which chunk(s) are actually relevant. This can be done lightly: annotate 20–30 rows with the specific page that proves or disproves compliance, then compute MRR on that subset.

Hit Rate@k

Priority: Important | **Status:** Not implemented | **Path:** Document path only

What it measures: Binary metric: is at least one relevant page present anywhere in the top-k retrieved chunks? Answers the yes/no question "did retrieval completely fail for this row?"

Why it matters in this domain: This is the simpler, more actionable sibling of Context Recall. A hit rate < 80% means the strategy structurally misses approximately 1 in 5 document requirements — a hard blocker for production use. It is easier to interpret than recall when communicating results to non-technical stakeholders (the SSI team).

Variants to track:

- `Hit Rate@4` — for strategies that retrieve 4 pages (naive_rag, bm25_hybrid, hyde)
- `Hit Rate@20` — for the reranker's BM25 first stage (checking whether the cross-encoder has anything good to work with)

To implement: Same annotation requirement as MRR: a small set of rows where the ground-truth relevant page is identified. A shared annotation effort of 2–3 hours covers both MRR and Hit Rate@k.

Context Entity Recall

Priority: Important | **Status:** Not implemented | **Path:** Document path only

What it measures: Whether the retrieved chunks contain the specific named entities that the expert cited as evidence: certificate numbers, issuer names, ISO standard codes, validity dates, and similar domain-specific identifiers.

Why it matters in this domain: A retrieved page that mentions "ISO 27001" in a generic context is not the same as a page that contains the specific certificate number, the issuer (Bureau Veritas, AFNOR, etc.), and the validity dates. Context entity recall catches the case where a page is superficially on-topic but factually mismatched — a common failure mode when the proof folder contains multiple versions of a certificate from different years.

Example: Expert note: "Certificat Bureau Veritas n° FR-2025-ISO27001-00847, valide jusqu'au 14/01/2026" Entity check: does the retrieved page contain "FR-2025-ISO27001-00847", "Bureau Veritas", and a date consistent with 2026?

To implement:

Maybe add a way to classify certificates or do something about topic modeling

1. Extract named entities from human_notes using spaCy (`fr_core_news_md` for French) or a regex pattern list for known compliance entities (ISO codes, cert number formats, date patterns)
2. Check presence of each entity in the retrieved pages
3. Report as fraction of expected entities found

Layer 2 — Generation quality

Faithfulness

Priority: Critical | **Status:** Implemented | **Path:** Document path only

What it measures: Whether every factual claim in the AI reasoning is actually supported by the retrieved context. Answers: is the reasoning grounded in the documents, or is the LLM generating from its parametric memory?

Why it matters in this domain: Hallucinated certificate numbers, incorrect expiry dates, or invented issuer names in the verdict reasoning are a direct compliance risk. The SSI expert team may rely on the reasoning text to confirm compliance without re-reading the source document. A faithfully grounded reasoning means every claim can be verified by reading the cited page.

Stanford AI Lab research indicates that poorly evaluated RAG systems can hallucinate in up to 40% of responses even when they have access to the correct information.

Current implementation: Token-level overlap between the AI reasoning text and the concatenated retrieved page texts. Serves as a lower bound — if overlap is very low, the reasoning is almost certainly drawing on parametric knowledge rather than retrieved content.

Upgrade path: RAGAS-style atomic claim decomposition: decompose the reasoning into individual statements ("the certificate was issued by Bureau Veritas", "the validity date is January 2026"), then use an LLM entailment check to verify each claim against the retrieved chunks. This is far more accurate than token overlap for French-language text with paraphrase.

Thresholds:

- ▌ 0.60: reasoning is largely grounded
- 0.40–0.60: some claims may be inferred rather than directly cited
- < 0.40: LLM is generating from parametric knowledge — review prompt and retrieved context quality

---

Reasoning Soft Match

Priority: Important | **Status:** Implemented | **Path:** Oral path only

What it measures: Token overlap between the AI reasoning and the expert's notes, applied to oral/declarative exigences where there are no documents to retrieve.

Why it matters in this domain: For requirements like "the employees received cybersecurity training in the last 12 months" or "the vendor commits to 72-hour incident notification", there are no documents to check (or the documents, if any, are incidental). The only way to evaluate reasoning quality is to compare the AI's logic against what an expert would have noticed and written.

Example: Expert note: "Attestation collective de formation fournie (oct. 2024, 45 collaborateurs). Contenu: phishing, RGPD, mots de passe." AI reasoning covers: "45 collaborateurs formés en octobre 2024" — soft match should be high. AI reasoning misses the training content details — soft match will be penalised.

Upgrade path: Replace token overlap with BERTScore using `camembert-base` or `xlm-roberta-base` for French text. BERTScore handles synonymy and paraphrase that token overlap misses entirely.

Hallucination Rate

Priority: Important | **Status:** Not implemented | **Path:** All paths

What it measures: The percentage of verdicts containing at least one factual claim that is not supported by the retrieved context or the representative's declaration. A more precise and actionable version of the faithfulness metric.

Why it matters in this domain: Faithfulness gives a continuous score. Hallucination rate gives a binary flag per verdict — useful for building a manual review queue. Any verdict where hallucination is detected should be routed to the SSI expert regardless of the AI's confidence score.

To implement: Use an LLM-as-judge second call: provide the AI reasoning and the retrieved pages, then prompt the judge to list any claims in the reasoning that cannot be directly verified in the context. A claim without support is counted as a hallucination.

RAGAS provides a ready-made implementation of this via its `faithfulness` evaluator that decomposes reasoning into atomic claims and performs entailment checking.

Target: < 5% hallucination rate for production use. Above 10% is a prompt engineering problem.

Citation Accuracy

Priority: Important | **Status:** Not implemented | **Path:** Document path only

What it measures: When the AI verdict cites a specific file ("see `certificat_iso27001_2025.pdf`, page 1"), does that file and page actually contain the evidence described?

Why it matters in this domain: The SSI expert team performs spot-checks on AI verdicts by pulling the cited document. A wrong citation — citing a file that was not retrieved, or citing page 3 when the evidence is on page 1 — destroys trust in the system immediately and in practice makes the tool unusable for routine verification.

To implement: This is a structural check, not an ML metric. Cross-check the `source_files` and `source_page` fields in the verdict dict against the `retrieved_pages` list:

1. Every file in `source_files` must appear in `retrieved_pages[i]["source_file"]`
2. The `source_page` value must match a page that was actually retrieved

This can be implemented as a post-processing validation step with zero additional LLM calls.

Layer 3 — Verdict / classification agreement

Weighted Cohen's Kappa

Priority: Critical | **Status:** Implemented | **Path:** All paths

What it measures: Agreement between the AI verdict and the human expert label, adjusted for chance, with ordinal penalties that reflect real-world error costs.

The penalty matrix:

```
True \ Predicted    | compliant | partially | non_compliant
-------------------|-----------|-----------|--------------
compliant           |     0     |     1     |       2
partially           |     1     |     0     |       1
non_compliant       |     2     |     1     |       0
```

A `compliant` ↔ `non_compliant` disagreement is penalised twice as heavily as an adjacent-class disagreement (`compliant` ↔ `partially_compliant`). This matches the actual cost structure of the compliance domain. Need to be understood completly

Why this is the primary metric: Weighted κ is the standard for ordinal clinical and compliance classification tasks — used in ICD medical coding, audit risk grading, credit rating, and legal compliance scoring. It is the only metric that simultaneously accounts for ordinal ordering and chance-corrected agreement.

Interpretation:

- κ > 0.80: excellent — system is a reliable first-pass filter
- κ 0.60–0.80: good — acceptable for assisted review with human override
- κ 0.40–0.60: moderate — may assist but requires high human oversight
- κ < 0.40: insufficient — do not use for compliance decisions

Implementation: `utils/evaluation.py` with the exact 3×3 penalty matrix. Used in all evaluation reports and the strategy comparison Overview sheet.

Research backing: Weighted κ is recommended by the ISACA AI Auditing framework for ordinal classification tasks in regulated industries. The LegalBench-RAG benchmark uses a similar ordinal agreement metric for compliance-related QA tasks.

Macro F1 + per-class F1

This really important to understand

Priority: Critical | **Status:** Implemented | **Path:** All paths

What it measures: Precision, recall, and F1 score computed separately for each of the three classes, then macro-averaged (equal weight per class regardless of class frequency).

Why per-class breakdown is essential: A model that predicts "partially_compliant" for every row achieves approximately 33% exact match and non-zero macro F1, but has zero recall on both `compliant` and `non_compliant`. Macro F1 alone hides this failure. Tracking `F1(non_compliant)` separately is non-negotiable.

Key insight for this domain: `non_compliant` is typically the least frequent class (most subcontractors are at least partially compliant). A model with high macro F1 but low `F1(non_compliant)` is systematically failing on the most important class. Set a hard floor: `F1(non_compliant) > 0.60` as an acceptance criterion independent of macro F1.

Implementation: `utils/evaluation.py`. Per-class and macro F1 reported in all evaluation sheets.

False Negative Rate on non-compliant

Priority: Critical | **Status:** Not implemented | **Path:** All paths

What it measures: The fraction of true `non_compliant` rows that the AI labels as `compliant` or `partially_compliant`. The single most important failure mode in this system.

Why it matters in this domain: This is the production acceptance criterion. A subcontractor with an expired PCA, a wrong-version charte SSI, or no signed clauses RGPD must not pass through the compliance gate labelled "compliant". The SSI expert team uses the AI output to prioritise manual review — if FNR(non_compliant) is high, risky subcontractors bypass human oversight.

Formula:

```
FNR(non_compliant) = FN_non / (FN_non + TP_non)
```

where `FN_non` = rows that are truly non-compliant but predicted as compliant or partial, and `TP_non` = rows that are truly non-compliant and correctly predicted as non-compliant.

Target threshold: < 10% for production deployment. The system should not miss more than 1 in 10 true non-compliant requirements.

To implement: Trivial — derive directly from the existing confusion matrix in `utils/evaluation.py`. Add as a dedicated metric row in the Overview sheet of the comparison report and as a named assertion in the evaluation suite.

Exact Match (strict accuracy)

Priority: Important | **Status:** Implemented | **Path:** All paths

What it measures: The fraction of rows where the AI label exactly matches the human expert label.

How to read it in this domain: Exact match is a useful sanity check but should never be the primary metric. The pathologies are:

- High exact match + low κ: impossible
- Low exact match + moderate κ: errors are all adjacent (compliant ↔ partial) — tolerable
- High exact match + low F1(non_compliant): model is accurate on easy cases but systematically misses the hard class

Use alongside κ. A system with 65% exact match and κ = 0.72 is production-ready. A system with 72% exact match and κ = 0.45 is not.

Layer 4 — Domain-specific additions

Confidence Calibration (ECE)

Priority: Critical | **Status:** Not implemented | **Path:** All paths

What it measures: Whether the model's confidence scores are actually predictive of its accuracy. A well-calibrated model that reports 0.90 confidence should be correct approximately 90% of the time.

Why it matters in this domain: SSI experts use the AI confidence score to decide whether a case requires manual deep-review or can be accepted as-is. Systematic overconfidence on wrong verdicts is the most dangerous failure mode: an expert will skip review of a 0.92-confidence "compliant" that is actually non-compliant.

How to measure — Expected Calibration Error (ECE):

1. Bucket all predictions by confidence: [0–0.5], [0.5–0.7], [0.7–0.85], [0.85–0.95], [0.95–1.0]
2. For each bucket, compute the mean confidence and the actual accuracy
3. ECE = weighted average of |mean_confidence − actual_accuracy| across buckets

A perfectly calibrated model has ECE = 0. ECE > 0.15 is a sign that confidence thresholds need recalibration.

Calibration fix: If the model is systematically overconfident, either:

- Add an explicit calibration instruction to the verdict prompt ("rate your confidence conservatively — 0.9 means you are certain beyond doubt")
- Apply post-hoc Platt scaling to the raw confidence scores

THis is the prolem i got with the first solution where when we dont have documents it always ends up as a partially compliant even when its compliant

Evidence Type Accuracy

Priority: Important | **Status:** Not implemented | **Path:** All paths (bm25_hybrid specifically)

What it measures: Whether the `evidence_mining` node in `bm25_hybrid` correctly classifies each exigence as `document_required`, `oral_implicit`, or `ambiguous`.

Why it matters in this domain: Evidence routing is the gate that determines whether retrieval happens at all. A misclassified oral requirement triggers full retrieval, which will always fail because no document proving "the vendor commits to X" exists in the proof folder. This inflates false negatives on oral requirements and contaminates the retrieval metrics for `bm25_hybrid`.

To implement: For this i might see the comments and do some nlp analysis to see the exigences that need a proof

1. Add a column to the input Excel: `Type_Preuve_Expert` (human annotation of expected evidence type for each row)
2. Compare against the AI's `Type_Preuve_Attendue` column (column L in the output Excel)
3. Report simple classification accuracy and a confusion matrix across the three evidence types

Target: > 85% accuracy on evidence type classification. Below this threshold, retrieval metrics for `bm25_hybrid` are not interpretable.

Overreliance on Irrelevant Documents

Priority: Important | **Status:** Not implemented | **Path:** Document path only

What it measures: Cases where the AI produces a high-confidence verdict based on retrieved pages that are clearly not relevant to the exigence. The failure mode specific to messy proof folders.

Why it matters in this domain: This is the most likely failure in production. A subcontractor submits a 80-page merged administrative dossier containing sections on cooking recipes, outdated insurance certificates, and financial statements alongside the relevant ISO certificate. The retrieval system may pull the wrong section, and the LLM — under context pressure — generates a plausible-sounding verdict about ISO 27001 by hallucinating around the irrelevant content.

Detection rule: Flag any verdict where all three conditions hold:

1. `confidence > 0.70` (AI is sure)
2. `context_precision < 0.30` (retrieved chunks are mostly irrelevant)
3. `source_files` cited do not overlap with `proof_refs` provided by the representative

These rows are high-risk false positives and should be automatically routed to mandatory human review.

To implement: Post-processing rule on the output verdict dict. No additional LLM calls required.

Latency by Evidence Type

Priority: Nice to have | **Status:** Implemented | **Path:** All paths

What it measures: Mean and P95 latency per row, broken down by evidence type (oral vs document) and by strategy.

Why it matters in this domain: The oral path (bm25_hybrid) skips retrieval entirely and should complete in 2–4 seconds per row. The document path with full BM25 + semantic search should take 10–20 seconds. HyDE adds one extra LLM call and will be the slowest strategy at 15–30 seconds per row. If the oral path is as slow as the document path, the evidence routing is broken.

Targets:

- Oral path: < 5s per row
- Document path (naive_rag, reranker, hyde): < 20s per row
- Document path (bm25_hybrid with keyword expansion): < 25s per row

Implementation: Done — the latency sheet in `results/comparison_report.xlsx` and per-row `latency_ms` in `StrategyResult`.

Explainability Score (EU AI Act)

Priority: Important | **Status:** Not implemented | **Path:** All paths

What it measures: The fraction of verdicts that include a traceable source citation — a specific file name, page number, or a direct quote from the representative's declaration.

Why it matters in this domain: The EU AI Act (Article 13) requires high-risk AI systems to provide "sufficient information" for human oversight of decisions. AXA's compliance verification decisions have regulatory downstream effects: a subcontractor approved or rejected based on this system must be re-reviewable. "The AI said non-compliant" is not a defensible audit trail.

Operationalisation:

- For `document_required` rows: the verdict must contain a non-empty, non-null `source_files` field pointing to a real file in the proof folder. Completeness rate target: 100%.
- For `oral_implicit` rows: the verdict reasoning must quote or paraphrase the specific representative declaration that supports the conclusion. This is harder to automate — a simple heuristic is to check that the representative's `rep_comments` text has at least 30% token overlap with the reasoning.

To implement: Completeness check post-processing on the verdict dict. Add as a column in the per-row evaluation sheet.

Metrics not applicable in this domain

The following metrics from general RAG evaluation frameworks are explicitly excluded and why:

Metric	Why excluded
BLEU / ROUGE	Designed for translation and summarisation with reference outputs. Compliance verdicts are not text generation tasks with a canonical output.
Answer Relevance (RAGAS)	Measures whether the answer addresses the query. The verdict always addresses the exigence. What matters is whether it is correct, not whether it is relevant.
Perplexity	A language model metric unrelated to factual accuracy or compliance correctness.
Exact match on generation text	Verdicts are paraphrase-rich and language-variable. Only the compliance label (the three-class categorical) should be exact-matched.
NDCG@k for ranking	Not applicable — this system does not return a ranked list of compliance items. All exigences are evaluated independently.
Task Completion Rate / User Satisfaction	Operational metrics for deployed conversational systems. Useful post-deployment but not for offline evaluation of retrieval and verdict quality.

Implementation roadmap

Already done (9 metrics)

- Context Recall
- Context Precision
- Faithfulness
- Reasoning Soft Match (oral path)
- Weighted Cohen's κ
- Macro F1 + per-class F1
- Exact Match
- Latency by evidence type
- Confusion matrix

Next sprint — high impact, low effort

These can be added to `utils/evaluation.py` with no new LLM calls:

1. **False Negative Rate on non-compliant** — one formula applied to the existing confusion matrix
2. **Citation Accuracy** — structural check on the verdict dict, no LLM needed

3. **Overreliance detection rule** — three-condition flag on existing fields
4. **Explainability completeness** — string length check on `source_files`

Next sprint — requires annotation

5. **Hit Rate@k** — annotate 20–30 rows with ground-truth relevant pages (~2 hours)
6. **MRR** — same annotation, derived for free
7. **Evidence Type Accuracy** — add `Type_Preuve_Expert` column to Excel, annotate when collecting expert feedback

Later — requires extra LLM calls

8. **Confidence Calibration (ECE)** — compute from existing confidence scores, plot calibration curve
9. **Hallucination Rate** — second LLM call per verdict to check atomic claims
10. **Context Entity Recall** — NER extraction from human_notes using spaCy `fr_core_news_md`

Metric selection by use case

Comparing strategies (runner.py --compare)

Question	Primary metric	Supporting metrics
Which strategy is most accurate?	Weighted κ	Macro F1, F1(non_compliant)
Which retrieves better?	Context Recall	Context Precision, Hit Rate@k
Which hallucinates less?	Faithfulness	Hallucination Rate
Which is safest for production?	FNR(non_compliant)	Confidence Calibration
Which is fastest?	Avg latency	Latency by evidence type

Evaluating a single run for production readiness

Gate	Threshold	Blocking?
Weighted κ	> 0.60	Yes
F1(non_compliant)	> 0.60	Yes
FNR(non_compliant)	< 10%	Yes

Gate	Threshold	Blocking?
Context Recall	> 0.75 on document rows	Yes
Faithfulness	> 0.55	Yes
Explainability completeness	100% on document rows	Yes
Exact Match	> 55%	No (informational)
Latency P95	< 30s per row	No (SLA, not quality)

Presenting results to SSI experts (non-technical audience)

Avoid raw metric values. Translate to:

- "The system correctly identified X% of non-compliant subcontractors"
- "When the system was confident (> 80%), it was correct Y% of the time"
- "On average, results were available in Z seconds per requirement"
- "Every verdict cites the specific document and page it relied on"